Part 0

Setup check showing all six PNG images found in the Images folder

```
from pathlib import Path
from PIL import Image, ExifTags

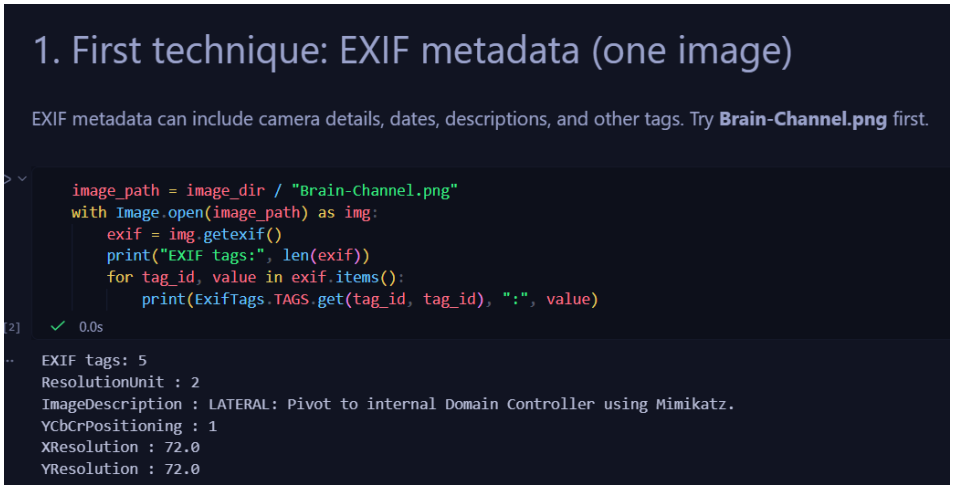
image_dir = Path("Images")
print("Images found:", len(list(image_dir.glob("*.png"))))
```

```
[1]  ✓ 0.0s

...  Images found: 6
```

Part 1

EXIF extraction for Brain-Channel.png



EXIF extraction for Activation.png showing 10 standard camera tags *and no message-carrying field*

```
image_path = image_dir / "Activation.png"
with Image.open(image_path) as img:
    exif = img.getexif()
    print("EXIF tags:", len(exif))
    for tag_id, value in exif.items():
        print(ExifTags.TAGS.get(tag_id, tag_id), ":", value)
```

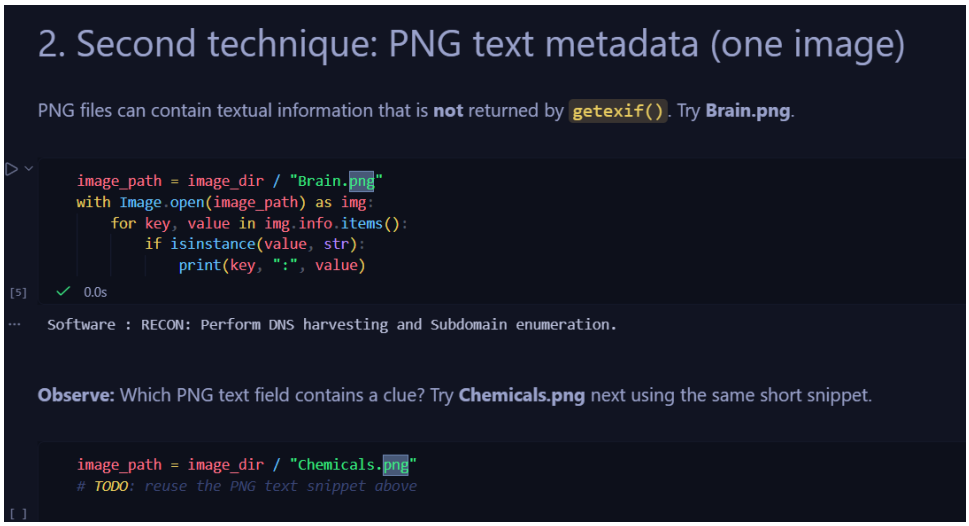
```
✓ 0.0s

EXIF tags: 10
GPSInfo : 396
ResolutionUnit : 2
ExifOffset : 204
Make : Nikon
Model : D750
Software : Adobe Lightroom 13.0
DateTime : 2025:02:02 10:15:00
YCbCrPositioning : 1
XResolution : 72.0
YResolution : 72.0
```

Evidence Log: Activation.png showed 10 EXIF camera tags with no message field, so EXIF is ruled out and its PNG text check is still to come. Brain-Channel.png showed 5 EXIF tags, and the ImageDescription field holds the fragment LATERAL: Pivot to internal Domain Controller using Mimikatz, so its channel is EXIF and no decoding is needed. The stage label in that fragment is LATERAL.

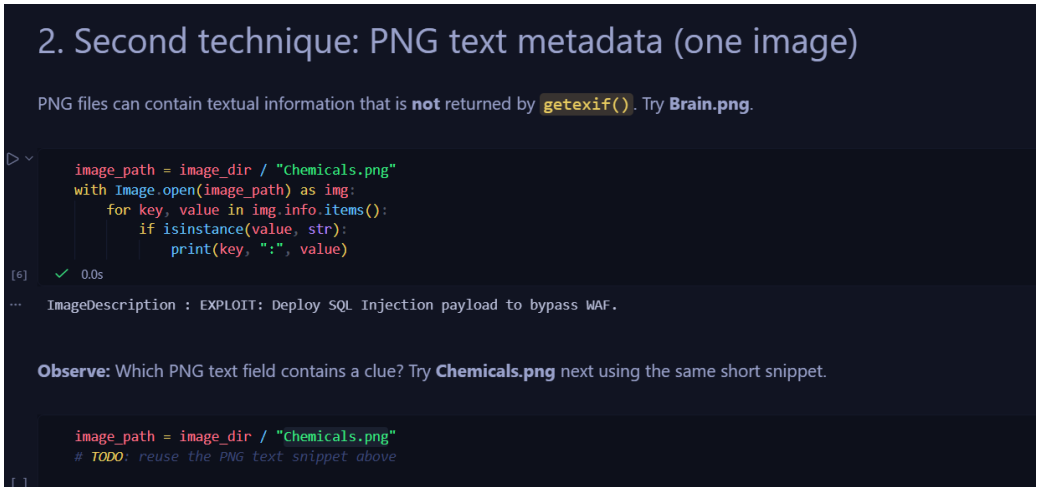
Try it yourself: No, zero EXIF tags does not mean the image has no metadata, because getexif only reads EXIF while PNG text and XMP are stored separately.

Part 2



The Software field in its PNG text holds the fragment RECON: Perform DNS harvesting and Subdomain enumeration, so its channel is PNG text and no decoding is needed. The stage label here is RECON.

Evidence so far: Activation.png showed 10 EXIF camera tags with no message field, and its PNG text check is still open. Brain-Channel.png gave its fragment through the EXIF ImageDescription field with the stage label LATERAL. Brain.png gave its fragment through the PNG text Software field with the stage label RECON.



The ImageDescription field in its PNG text holds the fragment EXPLOIT: Deploy SQL Injection payload to bypass WAF, so its channel is PNG text and no decoding is needed. The stage label here is EXPLOIT.

Activation.png showed 10 EXIF camera tags with no message field, and its PNG text check is still open. Brain-Channel.png gave its fragment through the EXIF ImageDescription field with the stage label LATERAL. Brain.png gave its fragment through the PNG text Software field with the stage label RECON. Chemicals.png gave its fragment through the PNG text ImageDescription field with the stage label EXPLOIT.

Part 3

XMP packet for Dopamine.png showing the exif:MakerNote element containing the partial secret.



The XML element is exif:MakerNote, and it holds PERSIST: Establish reverse shell via obfuscated cron job. The EXIF-only approach missed it because getexif only reads the EXIF segment, while XMP is a separate XML packet that Pillow exposes through img.info instead. That is also why the snippet checks both XML:com.adobe.xmp and xmp, since the key varies with the software that created the file.

Part 4



Getting the secret from Music.png which is in the copyright field



```
image_path = image_dir / "Music.png"
with Image.open(image_path) as img:
    exif = img.getexif()
    print("EXIF tags:", len(exif))
    for tag_id, value in exif.items():
        print(ExifTags.TAGS.get(tag_id, tag_id), ":", value)
```

[12] ✓ 0.0s

```
EXIF tags: 9
ResolutionUnit : 2
ExifOffset : 264
Make : Olympus
Model : OM-D E-M10
DateTime : 2025:02:06 11:11:00
YCbCrPositioning : 1
Copyright : RVhGSUw6IFR1bm5lbCBjb21wcmVzc2VkIGRhdGFiYXNlIGR1bXAgdmlhIERPUyBDb3ZlcnQgQ2hhbm5lbC4=
XResolution : 72.0
YResolution : 72.0
```

Base64 decoding of the Copyright value from Music.png revealing the partial secret



4. Fourth technique: Decode suspicious text (one ima

Sometimes metadata contains **encoded** rather than readable text. Try **Music.png**. First inspect its EXIF or P
this small example.

```
import base64

example = "RVhGSUw6IFR1bm5lbCBjb21wcmVzc2VkIGRhdGFiYXNlIGR1bXAgdmlhIERPUyBDb3ZlcnQgQ2hhbm5lbC4="
print(base64.b64decode(example).decode("utf-8"))
```

13] ✓ 0.0s

```
EXFIL: Tunnel compressed database dump via DNS Covert Channel.
```

Part 5

I adapted the Section 1 snippet because it only lists top-level EXIF tags and never opens the Exif sub-IFD. Activation's top-level EXIF, PNG text, and XMP checks showed only camera metadata or nothing at all, with no message field. The UserComment field sits in the Exif sub-IFD, so I used get_ifd to open that sub-block, which revealed the fragment. This follows the notebook's warning not to assume getexif shows all metadata.

```python
image_path = image_dir / "Activation.png"
with Image.open(image_path) as img:
    exif = img.getexif()
    print("=== Exif sub-IFD ===")
    try:
        exif_ifd = exif.get_ifd(0x8769)
        for tag_id, value in exif_ifd.items():
            print(ExifTags.TAGS.get(tag_id, tag_id), ":", value)
    except Exception as e:
        print("No Exif sub-IFD:", e)
    print("=== GPS IFD ===")
    try:
        gps_ifd = exif.get_ifd(0x8825)
        for tag_id, value in gps_ifd.items():
            print(ExifTags.GPSTAGS.get(tag_id, tag_id), ":", value)
    except Exception as e:
        print("No GPS IFD:", e)
```

[16] ✓ 0.0s

```
=== Exif sub-IFD ===
ExifVersion : b'0232'
ComponentsConfiguration : b'\x01\x02\x03\x00'
FlashPixVersion : b'0100'
DateTimeOriginal : 2025:02:02 10:15:00
DateTimeDigitized : 2025:02:02 10:15:00
ColorSpace : 65535
UserComment : b'ASCII\x00\x00\x00VULN: Identify unpatched CVE-2024-XXXX in web gateway.'
=== GPS IFD ===
GPSVersionID : b'\x02\x03\x00\x00'
GPSLatitudeRef : N
GPSLatitude : (40.0, 42.0, 46.08)
GPSLongitudeRef : W
GPSLongitude : (74.0, 0.0, 21.6)
```

Image	Metadata type / field	Raw Value	Decoding needed?	Partial Secret recovered
Activation.png	EXIF sub-IFD / UserComment	Bytes starting with an ASCII marker, followed by the fragment text	No, ASCII prefix stripped	VULN: Identify unpatched CVE-2024-XXXX in web gateway.
Brain-Channel.png	EXIF / ImageDescription	LATERAL: Pivot to internal Domain Controller using Mimikatz.	No	LATERAL: Pivot to internal Domain Controller using Mimikatz.

Brain.png	PNG text / Software	RECON: Perform DNS harvesting and Subdomain enumeration.	No	RECON: Perform DNS harvesting and Subdomain enumeration.
Chemicals.png	PNG text / ImageDescription	EXPLOIT: Deploy SQL Injection payload to bypass WAF.	No	EXPLOIT: Deploy SQL Injection payload to bypass WAF.
Dopamine.png	XMP / exif:MakerNote	<exif:MakerNote>PERSIST: Establish reverse shell via obfuscated cron job.</exif:MakerNote>	No	PERSIST: Establish reverse shell via obfuscated cron job.
Music.png	EXIF / Copyright	RVhGSUw6IFR1bm5lbCBjb21wcmVzc2VkIGRhdGFiYXNlIGR1bXAgdmlhIERPUyBDb3ZlcnQgQ2hhbm5lbC4=	Yes, Base64	EXFIL: Tunnel compressed database dump via DNS Covert Channel.

Part 6

1. Stage: RECON.
Recovered message: Perform DNS harvesting and Subdomain enumeration.

2. Stage: VULN.
Recovered message: Identify unpatched CVE-2024-XXXX in web gateway.

3. Stage: EXPLOIT.
Recovered message: Deploy SQL Injection payload to bypass WAF.

4. Stage: LATERAL.
Recovered message: Pivot to internal Domain Controller using Mimikatz.

5. Stage: PERSIST.
Recovered message: Establish reverse shell via obfuscated cron job.

6. Stage: EXFIL.
Recovered message: Tunnel compressed database dump via DNS Covert Channel.

Each stage depends on the one before it. Recon reveals the target surface, which lets the attacker identify the unpatched gateway vulnerability, which the SQL injection exploit then uses for the initial foothold. From there the attacker pivots to the Domain Controller, locks in access with the persistent reverse shell, and only then has the position needed to tunnel the database dump out. Any other order breaks this chain, since exfiltration without persistent access or lateral movement without a foothold would not work.

Step 7

Step 7A loop output confirming all six images in PNG format, with the **Step 7B** starter row for Brain.png showing its metadata fields.



Step 7A — Starter: loop through the six images.

```
for image_path in sorted(image_dir.glob("*.png")):
    with Image.open(image_path) as img:
        print(image_path.name, "→", img.format, "| fields:", len(img.info))
```

[17] ✓ 0.0s Python

```
Activation.png → PNG | fields: 8
Brain-Channel.png → PNG | fields: 4
Brain.png → PNG | fields: 4
Chemicals.png → PNG | fields: 1
Dopamine.png → PNG | fields: 5
Music.png → PNG | fields: 8
```

Step 7B — Starter: create one result row. Build on this structure rather than copying a complete solution.

```
image_path = image_dir / "Brain.png"
with Image.open(image_path) as img:
    row = {
        "filename": image_path.name,
        "format": img.format,
        "metadata_fields": ", ".join(img.info.keys()),
        "risk_score": 0,
    }
print(row)
```

[18] ✓ 0.0s Python

```
{'filename': 'Brain.png', 'format': 'PNG', 'metadata_fields': 'srgb, gamma, dpi, Software', 'risk_score': 0}
```

Step 7C The code, and scanner output for all six images, showing each suspected field, encoding, and risk score, with 6 rows saved to metadata_report.csv.

```
'''Decisions made
- The scanner checks metadata in the section 5 order. It checks EXIF
first, then the EXIF sub-IFD, then PNG text, then XMP.

- A field is flagged only when its name and its container match the
section 5 table. The list holds field names only, and no secret message
is hard coded.

- The same field name and value counted in two containers is counted
once. This follows the Lab 5 deduplication pattern.
```

```
- Base64 is checked only on the suspected value. The check uses validate
mode and requires readable text, and a successful decode is treated as an
indicator to investigate, not as proof.

- The score is 10 points for a suspected field, plus 5 points when that
value decodes from Base64 to readable text, plus 5 points only for a
second distinct suspected field. An encoded value is never also counted
as the additional indicator.

-A file that cannot be opened is recorded with a score of 0, and the loop
continues to the next image.
'''
import base64
import csv
from pathlib import Path

from PIL import ExifTags, Image

image_dir = Path("Images")

MESSAGE_FIELDS = {
    "EXIF": ("ImageDescription", "UserComment", "Copyright"),
    "EXIF sub-IFD": ("ImageDescription", "UserComment", "Copyright"),
    "PNG text": ("Software", "ImageDescription"),
    "XMP": ("MakerNote",),
}

def text_of(value):

    if isinstance(value, bytes):

        if value[:8] in (b"ASCII\x00\x00\x00", b"UNICODE\x00",
b"JIS\x00\x00\x00\x00"):
            value = value[8:]
        return value.decode("utf-8", errors="replace")
    return str(value)

def try_base64(value):

    try:
```

```
        decoded = base64.b64decode(value, validate=True).decode("utf-8")
    except Exception:
        return None
    if decoded.strip() and decoded.isprintable():
        return decoded
    return None

def collect_fields(img):

    collected = []

    exif = img.getexif()
    for tag_id, value in exif.items():
        name = ExifTags.TAGS.get(tag_id, tag_id)
        collected.append(("EXIF", str(name), text_of(value)))

    try:
        sub_ifd = exif.get_ifd(0x8769)
    except Exception:
        sub_ifd = {}
    for tag_id, value in sub_ifd.items():
        name = ExifTags.TAGS.get(tag_id, tag_id)
        collected.append(("EXIF sub-IFD", str(name), text_of(value)))

    for key, value in img.info.items():
        if isinstance(value, str):
            collected.append(("PNG text", key, value))

    xmp = img.info.get("XML:com.adobe.xmp") or img.info.get("xmp")
    if isinstance(xmp, bytes):
        xmp = xmp.decode("utf-8", errors="replace")
    if isinstance(xmp, str) and xmp:
        collected.append(("XMP", "XMP packet", xmp))

        start_tag = "<exif:MakerNote>"
        end_tag = "</exif:MakerNote>"
        start = xmp.find(start_tag)
        end = xmp.find(end_tag)
        if start != -1 and end != -1 and end > start:
            maker_note = xmp[start + len(start_tag):end]
```

```
            collected.append(("XMP", "MakerNote", maker_note))

    return collected

results = []

for image_path in sorted(image_dir.glob("*.png")):

    try:
        with Image.open(image_path) as img:
            image_format = img.format
            fields = collect_fields(img)
    except Exception as error:
        print(image_path.name, "could not be opened:", error)
        results.append({
            "filename": image_path.name,
            "format": "",
            "metadata_fields": "",
            "suspected_field": "",
            "encoding": "",
            "risk_score": 0,
            "reason": "File could not be opened, so no metadata was
checked.",
        })
        continue

    field_names = []
    for container, name, value in fields:
        if name not in field_names and name != "XMP packet":
            field_names.append(name)

    scored = set()
    risk_score = 0
    suspected = []
    encoding = ""
    reasons = []

    candidates = []
    seen_values = set()
    for container, name, value in fields:
```

```
        if (name, value) in seen_values:
            continue
        seen_values.add((name, value))
        if name in MESSAGE_FIELDS.get(container, ()) and value.strip():
            candidates.append((container, name, value))

    if candidates:
        container, name, value = candidates[0]
        suspected.append(container + " / " + name)
        if "covert-message field" not in scored:
            scored.add("covert-message field")
            risk_score += 10
            reasons.append(
                "Suspected covert-message field found in " + container
                + " field " + name + ", giving 10 points."
            )

        if try_base64(value) is not None:
            encoding = "Base64"
            if "encoded value" not in scored:
                scored.add("encoded value")
                risk_score += 5
                reasons.append(
                    "Suspected value decodes from Base64 to readable "
                    "text, giving 5 points for an encoded value."
                )
        else:
            encoding = "plain text"

        if len(candidates) > 1:
            second_container, second_name, second_value = candidates[1]
            suspected.append(second_container + " / " + second_name)
            if "additional indicator" not in scored:
                scored.add("additional indicator")
                risk_score += 5
                reasons.append(
                    "Second suspected field found in " + second_container
                    + " field " + second_name
                    + ", giving 5 points for an additional indicator."
```

```
                )
    else:
        reasons.append("No suspected message field found.")

    result = {
        "filename": image_path.name,
        "format": image_format,
        "metadata_fields": ", ".join(field_names),
        "suspected_field": "; ".join(suspected),
        "encoding": encoding,
        "risk_score": risk_score,
        "reason": " ".join(reasons),
    }
    results.append(result)

    print(
        result["filename"], "|", result["format"],
        "| suspected:", result["suspected_field"] or "none",
        "| encoding:", result["encoding"] or "none",
        "| score:", result["risk_score"],
    )

with open("metadata_report.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=results[0].keys())
    writer.writeheader()
    writer.writerows(results)

print("Saved", len(results), "rows to metadata_report.csv")
```

```
[19]  ✓ 0.0s

...  Activation.png | PNG | suspected: EXIF sub-IFD / UserComment | encoding: plain text | score: 10
     Brain-Channel.png | PNG | suspected: EXIF / ImageDescription | encoding: plain text | score: 10
     Brain.png | PNG | suspected: PNG text / Software | encoding: plain text | score: 10
     Chemicals.png | PNG | suspected: PNG text / ImageDescription | encoding: plain text | score: 10
     Dopamine.png | PNG | suspected: XMP / MakerNote | encoding: plain text | score: 10
     Music.png | PNG | suspected: EXIF / Copyright | encoding: Base64 | score: 15
     Saved 6 rows to metadata_report.csv
```

Filename	Suspected field	Encoding	Risk score	Reason

Activation.png	EXIF sub-IFD / UserComment	plain text	10	Suspected covert-message field found in EXIF sub-IFD field UserComment, giving 10 points.
Brain-Channel.png	EXIF / ImageDescription	plain text	10	Suspected covert-message field found in EXIF field ImageDescription, giving 10 points.
Brain.png	PNG text / Software	plain text	10	Suspected covert-message field found in PNG text field Software, giving 10 points.
Chemicals.png	PNG text / ImageDescription	plain text	10	Suspected covert-message field found in PNG text field ImageDescription, giving 10 points.
Dopamine.png	XMP / MakerNote	plain text	10	Suspected covert-message field found in XMP field MakerNote, giving 10 points.
Music.png	EXIF / Copyright	Base64	15	Suspected covert-message field found in EXIF field Copyright, giving 10 points. Suspected value decodes from Base64 to readable text, giving 5 points for an encoded

				value.

Brief interpretation
EXIF only extraction is limited because getexif reads only the EXIF segment. It misses PNG text fields, it misses the XMP packet, and its top level list misses values stored in the EXIF sub-IFD, such as UserComment. An image with no EXIF tags can therefore still carry a message in another container, and a complete check must cover all three containers.

Automated risk scoring is limited because the scores are heuristics, not probabilities. The field list comes from the section 5 investigation, so the scanner flags candidates for manual checking rather than proving a message on its own. An ordinary value in a flagged field, a duplicate value across containers, or a value that decodes cleanly can all change what a score means, which is why the section 5 table remains the authority and every flag still needs a human check.

Reflection Questions
1. Why can an image contain a hidden message even when getexif returns no tags?

getexif reads only the EXIF segment. PNG text chunks and the XMP packet are separate containers, and they are reached through img.info instead. A zero result therefore means the image has no EXIF tags, not that it has no metadata at all. The same point applies inside EXIF itself, since the top level list does not show every sub block.

2. How is encoding different from encryption? Is Base64 a secure way to hide information?

Encoding changes how data is represented using a public and reversible rule, and it uses no secret key. Encryption changes data using a key, so reversing it without the key is meant to be impractical. Base64 is encoding, not encryption, so it is not a secure way to hide information. Anyone who recognises a Base64 value can decode it, which is exactly what section 4 asks you to do.

3. Why should investigators preserve the original files and record hashes?

Preserving the originals keeps the evidence in the state it was found in, and working on copies prevents the investigation itself from changing it. A recorded hash gives a checkable fingerprint of that original state. If a later copy has the same hash, it has not changed. If the hash differs, the work cannot be trusted as a record of the original, and the result cannot be reproduced reliably.

4. Can metadata alone prove that an image was edited or that an incident occurred? What corroborating evidence would you need?

No, metadata alone cannot prove either point. Metadata can be edited, copied from another file, planted, or stripped, and ordinary software can add or change fields without any malicious activity. Corroboration would come from independent sources that line up with the metadata, such as file system timestamps, file hashes, system and application logs, network records, other files from the same device or time period, and the image content itself. A single metadata field is an indicator to investigate, not proof on its own.

5. How does this investigation connect to the automated evidence collection and analysis you practised in Labs 4 and 5?

Lab 4 collected packets in a loop, counted and grouped them, and flagged unusual patterns for review. Lab 5 looped over known endpoints, recorded one dictionary per request, flagged indicators rather than confirmed vulnerabilities, avoided duplicate alerts, and exported the results to a CSV file. Section 7 follows the same pattern with a different evidence source. It loops over images, extracts EXIF, PNG text, and XMP data, records one row per image, flags a suspected field, applies a simple score, and exports metadata_report.csv for manual checking against the section 5 table.